Implementation of the Extended Euclidean Algorithm, Modular Inverse, and Affine Cipher in Python

ARBAOUI HIBAT ALLAH

Contents

1 Introduction **2**

2 Mathematical Background **2**
2.1 1. Extended Euclidean Algorithm 2
2.2 2. Modular Inverse 2
2.3 3. Affine Cipher 3

3 Commented Source Code **3**
3.1 Class: BigIntMath 3
3.2 Class: AffineCipher 4

4 Execution Examples **5**
4.1 Test 1: Extended Euclidean Algorithm 5
4.2 Test 2: Modular Inverse 6
4.3 Test 3: Affine Cipher 6

1 Introduction

This project implements three fundamental concepts in number theory and classical cryptography:

- The Extended Euclidean Algorithm
- Modular Inverse computation
- The Affine Cipher

These concepts are strongly connected. The Extended Euclidean Algorithm allows us to compute the modular inverse, which is required to decrypt messages encrypted using the affine cipher.

2 Mathematical Background

2.1 1. Extended Euclidean Algorithm

The classical Euclidean algorithm computes:

$$\gcd(a, b)$$

The extended version additionally finds integers u and v such that:

$$au + bv = \gcd(a, b)$$

This is called **Bézout's Identity**.
If:

$$\gcd(a, n) = 1$$

then there exist integers u and v such that:

$$au + nv = 1$$

Reducing modulo n:

$$au \equiv 1 \pmod{n}$$

Therefore, u is the modular inverse of a modulo n.

2.2 2. Modular Inverse

The modular inverse of a modulo n is an integer x such that:

$$ax \equiv 1 \pmod{n}$$

It exists if and only if:

$$\gcd(a, n) = 1$$

2.3 3. Affine Cipher

The affine cipher is defined by:

$$E(m) = (am + b) \mod n$$

Decryption requires the modular inverse of a:

$$D(c) = a^{-1}(c - b) \mod n$$

Thus, the affine cipher depends directly on modular inversion, which itself depends on the extended Euclidean algorithm.

3 Commented Source Code

3.1 Class: BigIntMath

```
class BigIntMath:

    @staticmethod
    def extendedGCD(a, b):
        """
        Extended Euclidean Algorithm.

        Returns:
            d = gcd(a, b)
            u, v such that:
                a*u + b*v = d
        """

        # Initialize remainders
        r, r_prime = a, b

        # Initialize Bezout coefficients
        u, v = 1, 0
        u_prime, v_prime = 0, 1

        # Main loop
        while r_prime != 0:

            # Integer division
            q = r // r_prime

            # Update remainders
            r, r_prime = r_prime, r - q * r_prime

            # Update coefficients
            u, u_prime = u_prime, u - q * u_prime
            v, v_prime = v_prime, v - q * v_prime

        return r, u, v
```

```
    @staticmethod
    def modInverse(a, n):
        """
␣␣␣␣␣␣␣␣Computes␣the␣modular␣inverse␣of␣a␣modulo␣n.
␣␣␣␣␣␣␣␣"""

        d, u, _ = BigIntMath.extendedGCD(a, n)

        # Inverse exists only if gcd = 1
        if d != 1:
            raise ValueError(
                f"{a}␣is␣not␣invertible␣modulo␣{n}␣(gcd={d})"
            )

        # Ensure positive result
        return u % n
```

3.2 Class: AffineCipher

```
class AffineCipher:

    def __init__(self, a, b, alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ")
       :

        self.alphabet = alphabet
        self.n = len(alphabet)
        self.a = a
        self.b = b

        # Check that a and n are coprime
        d, _, _ = BigIntMath.extendedGCD(a, self.n)
        if d != 1:
            raise ValueError(
                "Parameter␣'a'␣must␣be␣coprime␣with␣alphabet␣size"
            )

        # Compute modular inverse of a
        self.a_inv = BigIntMath.modInverse(a, self.n)


    def encrypt(self, message):
        """
␣␣␣␣␣␣␣␣Encrypts␣a␣message␣using␣affine␣transformation.
␣␣␣␣␣␣␣␣"""

        result = ""

        for char in message.upper():
```

```
            if char in self.alphabet:

                # Convert letter to index
                m = self.alphabet.index(char)

                # Apply affine formula
                c = (self.a * m + self.b) % self.n

                result += self.alphabet[c]

            else:
                # Keep spaces and punctuation
                result += char

        return result


    def decrypt(self, ciphertext):
        """
        Decrypts an affine-encrypted message.
        """

        result = ""

        for char in ciphertext.upper():

            if char in self.alphabet:

                c = self.alphabet.index(char)

                # Apply inverse transformation
                m = (self.a_inv * (c - self.b)) % self.n

                result += self.alphabet[m]

            else:
                result += char

        return result
```

4 Execution Examples

4.1 Test 1: Extended Euclidean Algorithm

Input:

$$a = 56, \quad b = 11$$

Output:

$$gcd(56, 11) = 1$$

$$56u + 11v = 1$$

This confirms Bézout's identity.

4.2 Test 2: Modular Inverse

Input:

$$a = 7, \quad n = 26$$

Output:

$$7^{-1} \equiv 15 \pmod{26}$$

Verification:

$$7 \times 15 \equiv 1 \pmod{26}$$

4.3 Test 3: Affine Cipher

Parameters:

$$a = 5, \quad b = 8$$

Message:

$$BONJOUR$$

Encrypted message:

$$NWVRWAP$$

Decrypted message:

$$BONJOUR$$

Verification:

$$D(E(m)) = m$$